

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Vorlesung 9

Datum, Zeit und Internationalisierung

9.1 Datum und Zeit

Datum und Uhrzeiten sind wichtige Dinge in fast allen PHP-Applikationen. Sei es, um den Zeitpunkt eines Forenpostings zu kennzeichnen, die genaue Zeit zu der ein*e User*in das letzte Mal angemeldet war, die Berechnung von Zeiträumen, um eine Online-Reservierung vorzunehmen oder einfach nur, um die aktuelle Uhrzeit anzuzeigen. Die Arbeit mit Datum und Uhrzeit ist jedoch eine komplexere Thematik als einfach nur die Ausgabe von Zahlen und Zeichenketten. Verschiedene Zeitzonen, Zeit- und Datumsformate, Schaltjahre, Sommer- und Winterzeit, die Addition von Stunden zu einer Uhrzeit, die einen Tageswechsel zur Folge haben – kurzum, viele Dinge machen eine händische Berechnung und Verwaltung von Datum und Uhrzeit zur Herausforderung.

Daher sollte unbedingt auf die von PHP bereitgestellten Klassen `DateTime` (zur Verwaltung von Datum und Uhrzeit), `DateInterval` (zur Verwaltung von Zeiträumen) und `DateTimeZone` (zur Arbeit mit Zeitzonen) zurückgegriffen werden.

9.1.1 Datum und Zeit ausgeben

Um ein Datum und/oder die Zeit auszugeben, wird also immer ein Objekt der Klasse `DateTime` benötigt. Wird dieses ohne jegliche Parameter angelegt, so enthält es das aktuelle Datum und die Uhrzeit:

```
1 $d = new DateTime();
```

Um diese auszugeben, enthält das Objekt die Methode `format($format)`¹, die unter Angabe eines bestimmten Formats diese ausgibt. Dabei können die Angaben der Funktion `date()`² verwendet werden, Tabelle 9.1 gibt einen Auszug über einige Formatparameter.

Das aktuelle Datum und die Uhrzeit werden zunächst mit der Option `r` ausgegeben, dann mit der Option `d.m.Y, H:i:s e`. Das Ergebnis ist in Abbildung 9.1 zu sehen.

```
1 echo "<p>It's now {$d->format('r')}</p>";  
2 echo "<p>It's now {$d->format('d.m.Y, H:i:s e')}</p>";
```

¹<https://www.php.net/manual/de/datetime.format.php>

²<https://www.php.net/manual/de/function.date.php>

Tabelle 9.1: Einige Formatzeichen zur Datums- und Uhrzeitformatierung. Die volle Auflistung findet sich unter <https://www.php.net/manual/de/datetime.format.php>.

Zeichen	Beschreibung	Beispielwerte
d	Tag numerisch, mit führender Null	01–31
j	Tag numerisch, ohne führende Null	1–31
l	Wochentag textuell, ausgeschrieben	Monday–Sunday
N	Wochentag numerisch	1 (Montag)–7 (Sonntag)
m	Monat numerisch mit führender Null	01–12
F	Monat textuell, ausgeschrieben	January–December
Y	Jahr, vierstellig	2026, 1975
L	Schaltjahr oder nicht	1 (ja), 0 (nein)
G	Stunde, 24 h, ohne führende Null	0–23
g	Stunde, 12 h, ohne führende Null	1–12
H	Stunde, 24 h, mit führender Null	00–23
h	Stunde, 12 h, mit führender Null	01–12
i	Minute, mit führender Null	00–59
s	Sekunde, mit führender Null	00–59
e	Zeitzonebezeichner	UTC, GMT, Europe/Vienna
c	Vollständiges Datum/Uhrzeit, ISO 8601	2026-06-02T10:30:00+02:00
r	Vollständiges Datum/Uhrzeit, RFC 2822	Tue, 02 Jun 2022 10:30:00+0200

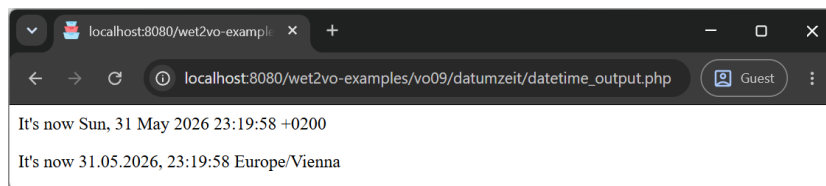


Abbildung 9.1: Die aktuelle Zeit wird einmal mit der Option **r**, dann mittels **d.m.Y, H:i:s e** ausgegeben.

9.1.2 Zeitzonen setzen

Unter Umständen wird im vorigen Beispiel nicht wie in Abbildung 9.1 „Europe/Vienna“, sondern „UTC“ (bzw. „+0000“ statt „+0200“), „Europe/Helsinki“ oder auch „America/Los_Angeles“ angezeigt. PHP übernimmt standardmäßig die am Webserver eingestellte Zeitzone, sofern diese nicht in der Datei `php.ini` explizit definiert ist. Steht dieser Webserver nun nicht in der lokalen Zeitzone oder seine Zeitzone ist einfach falsch konfiguriert, kommt es zur Ausgabe einer anderen bzw. unerwarteten Zeitzone.

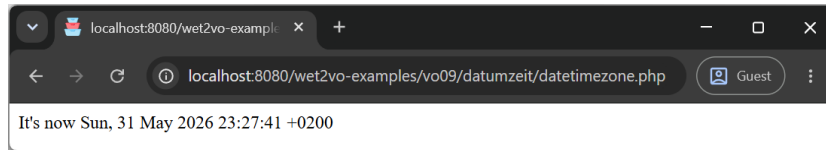


Abbildung 9.2: Der aktuelle Zeitstempel wird Dank `DateTimeZone`-Objekt in der richtigen Zeitzone dargestellt.

Standard-Zeitzone setzen

Dies kann auf zwei Arten gelöst werden. Zunächst kann die PHP-Konfiguration des aktuellen Webservers in der Datei `php.ini` mit folgender Konfiguration auf die für Österreich geltende Zeitzone gesetzt werden:

```
date.timezone = "Europe/Vienna"
```

Besteht keine Möglichkeit, dies bereits dort zu ändern, kann es auch zur Laufzeit im PHP-Code mit der Funktion `date_default_timezone_set()`³ gesetzt werden:

```
1 date_default_timezone_set("Europe/Vienna");
```

Eine Auflistung aller gültigen Zeitzone-Bezeichner findet sich unter <https://www.php.net/manual/de/timezones.php>.

Zeitzone verwalten

Oftmals ist es jedoch sinnvoll, UTC als Standard-Zeitzone anzunehmen bzw. zu belassen und alle Datums- und Zeitinformationen damit abzuspeichern (in Dateien oder einer Datenbank). UTC beachtet keine Sommer- und Winterzeit und kann als kleinster gemeinsamer Nenner oder Ausgangspunkt gesehen werden, wenn eine Applikation mit Daten aus verschiedenen Zeitzone arbeiten muss. In diesem Fall ist es sinnvoller, die Zeitzone erst bei der Anzeige von Daten der User*innen zu setzen.

Dafür existiert die Klasse `DateTimeZone`, die Zeitzone im Zusammenhang mit der `DateTime`-Klasse repräsentiert. Objekte dieser Klasse werden wiederum mit den gängigen Zeitzone-Bezeichnern als Argument erzeugt.

Sie können dann etwa als zweites Argument beim Konstruktor eines `DateTime`-Objekts angegeben werden. So wird die Zeitzone nur für dieses eine Objekt und nicht global im PHP-System gesetzt. Im folgenden Beispiel wird genau dies gemacht, Abbildung 9.2 zeigt das Ergebnis.

```
1 $timezone = new DateTimeZone("Europe/Vienna");
2 $d1 = new DateTime("now", $timezone);
3 echo "<p>It's now {"$d1->format("r")}</p>";
```

Alternativ kann die Zeitzone auch mit der Methode `setTimeZone($timezone)`⁴ nachträglich gesetzt werden. Eine zuvor festgelegte Zeit wird dabei entsprechend angepasst. Im folgenden Beispiel wird ein `DateTime`-Objekt mit einer bestimmten Datums-

³<https://www.php.net/manual/de/function.date-default-timezone-set.php>

⁴<https://www.php.net/manual/de/datetime.settimezone.php>

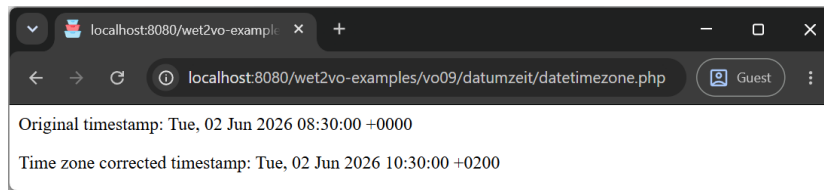


Abbildung 9.3: Ein Zeitstempel wird mit UTC angelegt und ausgegeben. Danach wird eine neue Zeitzone gesetzt und entsprechend dargestellt.

und Uhrzeitkombination in UTC angelegt. Dies könnte so etwa nach dem Einlesen eines (in einer Datei oder Datenbank) gespeicherten Eintrags der Fall sein. Vor der Ausgabe mit `format()` wird eine Zeitzone gesetzt. Das Ergebnis ist in Abbildung 9.3 ersichtlich.

```
1 $timezone = new DateTimeZone("Europe/Vienna");
2 $d2 = new DateTime("2026-06-02T08:30:00+00:00");
3 echo "<p>Original timestamp: {$d2->format("r")}</p>";
4 $d2->setTimezone($timezone);
5 echo "<p>Time zone corrected timestamp: {$d2->format("r")}</p>";
```

9.1.3 Mit Datum und Zeit rechnen

Ein `DateTime`-Objekt repräsentiert einen bestimmten Zeitpunkt. Oft ist es jedoch nötig, ausgehend von diesem einen neuen Zeitstempel zu berechnen. Dazu müssen bestimmte Zeitintervalle addiert oder subtrahiert werden, etwa „jetzt in 15 Stunden“ oder „heute vor zwei Wochen“. Diese Intervalle werden mit Hilfe der `DateInterval`-Klasse definiert.

Intervalle definieren

Um einen Zeitintervall festzulegen, wird ein Objekt der Klasse instanziiert. Als Argument wird ein String übergeben, der das Intervall festlegt. Er beginnt immer mit dem Buchstaben P für „Period“, gefolgt von einem Integerwert, der eine Dauer angibt. Abschließend wird ein Bezeichner gesetzt, der angibt, ob es sich um Tage, Wochen, Monate oder Jahre handelt. Werden Zeiten angegeben, werden diese durch den Buchstaben T getrennt spezifiziert. Tabelle 9.2 gibt die möglichen Bezeichner wieder.

Im folgenden Code werden drei Intervalle erstellt:

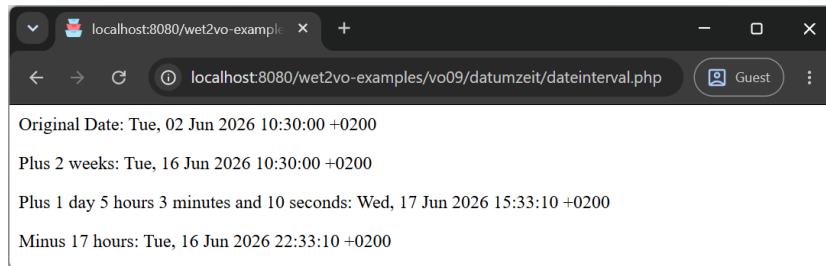
- zwei Wochen,
- ein Tag, fünf Stunden, drei Minuten und zehn Sekunden und
- 17 Stunden.

```
1 $twoWeeks = new DateInterval("P2W");
2 $oneDay5Hours3Mins10Secs = new DateInterval("P1DT5H3M10S");
3 $seventeenHours = new DateInterval("PT17H");
```

Zu beachten ist, dass bei der Instanziierung von `DateInterval`-Objekten eine Exception geworfen wird, falls der Intervallbezeichner nicht korrekt interpretiert werden kann. Es empfiehlt sich hier einen `try/catch`-Block zu verwenden, um dieser Fehlermöglichkeit zu begegnen.

Tabelle 9.2: Gültige Periodenbezeichner für `DateInterval`-Objekte.

Periodenbezeichner	Beschreibung
Y	Jahre
M	Monate
D	Tage
W	Wochen
H	Stunden
M	Minuten
S	Sekunden
F	Mikrosekunden

**Abbildung 9.4:** Zu einem Datum werden mehrere Intervalle addiert bzw. subtrahiert.

Intervalle addieren und subtrahieren

Mit Hilfe der Methoden `add($interval)`⁵ und `sub($interval)`⁶ können nun diese Intervalle zu einem `DateTime`-Objekt addiert bzw. davon subtrahiert werden. Das folgende Beispiel addiert zu einem Datum `$d` zuerst die ersten beiden Intervalle und subtrahiert dann den dritten. Abbildung 9.4 zeigt das Ergebnis.

```
1 echo "<p>Original Date: {$d->format("r")}</p>";
2 echo "<p>Plus 2 weeks: {$d->add($twoWeeks)->format("r")}</p>";
3 echo "<p>Plus 1 day 5 hours 3 minutes and 10 seconds:
    {$d->add($oneDay5Hours3Mins10Secs)->format("r")}</p>";
4 echo "<p>Minus 17 hours: {$d->sub($seventeenHours)->format("r")}</p>";
```

Zeitdifferenzen berechnen

Um die Differenz zwischen zwei `DateTime`-Objekten zu berechnen, steht die Methode `diff($datetime2)`⁷ zur Verfügung. Das Ergebnis ist ein `DateInterval`-Objekt.

⁵<https://www.php.net/manual/de/datetime.add.php>

⁶<https://www.php.net/manual/de/datetime.sub.php>

⁷<https://www.php.net/manual/de/datetime.diff.php>

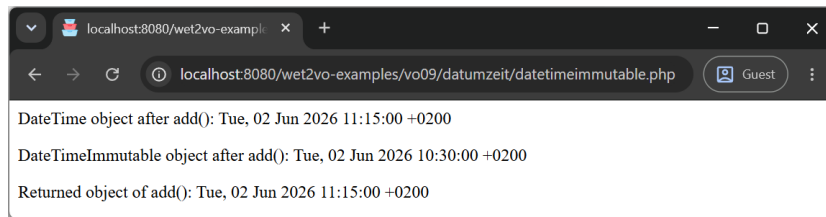


Abbildung 9.5: Ein `DateTimeImmutable`-Objekt wird nicht durch Intervall-Operationen verändert.

9.1.4 Unveränderbare Zeitstempel

Objekte der Klasse `DateTime` werden, wie in Abschnitt 9.1.3 demonstriert, verändert, wenn Zeitintervalle hinzugefügt werden. Dabei geht der ursprüngliche Zeitstempel verloren. Dies kann nicht erwünscht, oder sogar problematisch sein. Aus diesem Grund gibt es alternativ dazu die Klasse `DateTimeImmutable`. Sie bietet dieselben Methoden wie `DateTime`, jedoch sind Objekte der Klasse „immutable“, also unveränderbar. Wird eine Methode wie `add()` aufgerufen, wird also nicht das bestehende Objekt verändert, sondern ein neues Objekt erzeugt und zurückgegeben.

```
1 $d1 = new DateTime("2026-06-02T10:30:00+02:00");
2 $d2 = new DateTimeImmutable("2026-06-02T10:30:00+02:00");
3 $fortyFiveMinutes = new DateInterval("PT45M");
4
5 $d1->add($fortyFiveMinutes);
6 $d3 = $d2->add($fortyFiveMinutes);
7
8 echo "<p>DateTime object after add(): {$d1->format("r")}</p>";
9 echo "<p>DateTimeImmutable object after add(): {$d2->format("r")}</p>";
10 echo "<p>Returned object of add(): {$d3->format("r")}</p>";
```

In diesem Beispiel werden ein `DateTime`- und ein `DateTimeImmutable`-Objekt angelegt. Zu beiden werden 45 Minuten hinzugezählt und das Resultat wird ausgegeben. Wie in Abbildung 9.5 ersichtlich, ändert sich das `DateTime`-Objekt, das `DateTimeImmutable`-Objekt bleibt jedoch gleich. Um hier den neu entstandenen Zeitstempel (+45 Minuten) zu erhalten, muss der Rückgabewert von `add()` verwendet werden.

9.1.5 Erweiterte Funktionalitäten mit Carbon

Während die mit PHP mitgelieferten Klassen gute Basisfunktionalität für die Arbeit mit Datum und Zeit bieten, gibt es Fälle, in denen noch zusätzliche Funktionalität benötigt wird. Beispiele dafür könnten das Erstellen von Zeitstempeln aus verschiedensten Formaten, das einfachere Berechnen von Differenzen oder eine Lokalisierung von Monatsnamen und Wochentagen sein.

All diese Dinge und noch viel mehr bietet die PHP-Komponente `nesbot/carbon`⁸. Sie erweitert die `DateTime`-Klasse und bietet daher all ihre Funktionalitäten und vieles mehr. Carbon kann einfach über Composer installiert werden:

```
1 composer require nesbot/carbon
```

⁸<https://carbon.nesbot.com/>

Eine ausführliche Dokumentation aller Funktionen bietet die Carbon-Webseite.

9.2 Internationalisierung

Genauso wie beinahe jede PHP-Applikation Datum und Uhrzeit benötigt und dabei auf die unterschiedlichen Gegebenheiten wie Formate und Zeitzonen Rücksicht nehmen muss, so muss diese Anpassung an lokale Gegebenheiten meist noch eine Stufe höher gedacht werden. PHP-Applikationen laufen nicht nur in unterschiedlichen Zeitzonen, sondern werden von Benutzer*innen in unterschiedlichen Regionen, mit unterschiedlichen Sprachen und Schriften oder Kulturen verwendet. Dies muss bei der Erstellung der Applikation berücksichtigt werden, sollte diese global verwendet werden. Dieser Prozess wird *Internationalisierung* genannt.

9.2.1 Allgemeines zu Internationalisierung

Das W3C definiert *Internationalisierung* wie folgt [1]:

Internationalisierung entspricht dem Design und der Entwicklung eines Produkts, einer Anwendung oder eines Dokuments, das leichte Lokalisierung für Zielgruppen, die in Kultur, Region oder Sprache variieren, ermöglicht.

Internationalisierung, (engl. „internationalization“, auch mit „i18n“ – der Anzahl der Buchstaben zwischen „i“ und „n“ – abgekürzt) ermöglicht es also, in eine Applikation auf die Bedürfnisse von Nutzer*innen mit unterschiedlichen, so genannten *Locales* Rücksicht zu nehmen.

Locales

Eine Locale fasst die wichtigsten Internationalisierungs-Parameter für eine Region zusammen. Dazu gehören u. a.:

- die Sprache und Schrift (inklusive Leserichtung),
- das Land,
- das Datums- und Zeitformat,
- das Währungsformat.

Eine Locale wird in PHP (und anderen Sprachen) meist als IETF BCP 47 (RFC 5646) [2] Sprachtags angegeben. Diese setzen sich aus einer Reihe von Subtags zusammen, die durch einen Bindestrich getrennt sind. Begonnen wird mit dem verpflichtenden Subtag für die Primärsprache, danach können verschiedene Zusatzinformationen optional angegeben werden. Dazu gehören etwa Subtag für die Region oder ein Subtag für das verwendete Schriftsystem. Je genauer die Locale durch die Subtags beschrieben wird, desto spezifischer ist sie auf regionale Gegebenheiten angepasst. **de** bedeutet etwa Deutsch und umfasst alle deutschsprachigen Länder, während **de-AT** spezifisch österreichisches Deutsch kennzeichnet. Tabelle 9.3 gibt einen Überblick über einige solche Sprachtags.

Tabelle 9.3: Einige Locales in Form von IETF BCP 47 Sprachtags.

Sprachtag	Bedeutung
de	Deutsch
de-AT	Österreichisches Deutsch
de-DE	Standard-Deutsch (Deutschland)
en-US	US-Englisch
fr-CA	Kanadisches Französisch
fr-FR	Standard-Französisch (Frankreich)
zh-Hant-TW	Chinesisch, Han traditional, für Taiwan
sr-Cyrl	Serbisch, kyrillisch

Zum Arbeiten mit Locales stellt PHP die Klasse `Locale`⁹ zur Verfügung. Sie ermöglicht es, Sprachtags zu validieren und korrekt zurückzugeben.

Locales bilden also die Eigenschaften ab, die für die Internationalisierung einer Applikation wichtig sind. Der Prozess der Anpassung der Applikation an diese Eigenschaften wird nun Lokalisierung genannt.

Lokalisierung

Wie in der Definition am Beginn des Abschnitts angegeben, ermöglicht Internationalisierung die Lokalisierung eines Produktes oder eine Applikation. *Lokalisierung* (engl. „localization“ oder kurz „l10n“) ist also der tatsächliche Prozess der Angleichung einer Applikation an lokale Gegebenheiten. Also die Übersetzung in die Landessprache und das dabei verwendete Schriftsystem, die Angleichung an die verwendeten Datums-, Zeit- und Währungsformate oder auch der sorgfältige sprachliche Umgang mit kulturellen Eigenschaften. Lokalisierung kann also als die konkrete Anwendung von Internationalisierung gesehen werden.

Die folgenden Abschnitte zeigen Herangehensweisen in PHP, die dabei helfen sollen, die grundsätzlichen Abläufe von Lokalisierungen zu verstehen und sie einsetzen zu können. Dazu sollten aber die Herausforderungen, denen man als Entwickler*in dabei begegnet, bekannt sein.

Herausforderungen bei Internationalisierung und Lokalisierung

Bei der Internationalisierung und Lokalisierung von Anwendungen sollten sich Entwickler*innen der Tragweite dieses Prozesses bewusst sein. Es gibt weltweit um die 7.000 Sprachen in mehreren hundert Sprachfamilien. Allein der Prozess des Schreibens wurde auf der Welt vier Mal unabhängig voneinander auf verschiedene Arten und Weisen erfunden: von den Sumerern, den Ägyptern, den Chinesen und den Mayas. Dabei entstanden verschiedene Schriftsysteme. Ausgehend von den ägyptischen Hieroglyphen entwickelte

⁹<https://www.php.net/manual/de/class.locale.php>

sich etwa das lateinische Alphabet, kyrillisch, arabisch, oder die runische Schrift. Im asiatischen Raum erfolgte eine völlig andere Entwicklung, hier haben Schriftzeichen Bedeutungen. Diese (sehr grobe) Zusammenfassung alleine macht bereits deutlich, dass nicht einfach nur von „Sprache und Schrift“ gesprochen werden kann, sondern dass diese Dinge für Personen in unterschiedlichen Regionen der Erde auch eine völlig unterschiedliche Bedeutung und Herangehensweisen haben. Internationalisierung ist also nicht nur eine technische Herausforderung – es ist vor allem wichtig, sich vor Augen zu führen, dass es diese Unterschiede gibt und sie bei der Entwicklung von Applikationen zu bedenken.

Berücksichtigt man diese Vielfalt in Sprache und Schrift und versucht sie auf den Kontext der Internationalisierung von Softwareapplikationen herunterzubrechen, ergeben sich eine Reihe von Herausforderungen:

- *Geschlechtsspezifische Konjugationen* bei Verben und Adjektiven. Bei Sprachen wie Spanisch oder Italienisch werden etwa Adjektive an das Geschlecht des Hauptworts angepasst.
- *Pluralbildungen*. In deutsch und anderen romanischen Sprachen repräsentiert ein Gegenstand die Einzahl, während kein Gegenstand oder mehrere Gegenstände als Pluralform wiedergegeben werden (Ich habe eine Tomate. Ich habe keine Tomaten. Ich habe fünf Tomaten.). Andere Sprachen (z. B. slawische Sprachen) haben jedoch noch weitere Formen der Pluralbildung (z. B. unter 5 vs. über 5 etc.).
- *Datums-, Zeit- und Währungsformatierung*. Tag, Monat und Jahr werden etwa in unterschiedlicher Reihenfolge angegeben und haben in den jeweiligen Sprachen andere Namen. Darüber hinaus gibt es Länder, die eigene Kalendersysteme haben (z. B. der Iran). Auch Währungen werden in Ländern unterschiedlich dargestellt.
- *Lese- und Schreibrichtung*. Viele europäische Sprachen werden von links nach rechts und von oben nach unten (LTR, TTB – left to right, top to bottom) gelesen. Sprachen wie Hebräisch oder Arabisch hingegen werden von rechts nach links und von oben nach unten (RTL, TTB) gelesen. Im Chinesischen und Japanischen gibt es Texte, die zuerst von oben nach unten und dann von rechts nach links (TTB, RTL) gelesen werden.

Diese und noch viele Herausforderungen müssen mitbedacht werden. Die in den folgenden Abschnitten vorgestellten Lösungen werden einige dieser Themen dabei aufgreifen.

9.2.2 Internationalisierung und Lokalisierung mit PHP

Um nun Applikationen mit PHP zu internationalisieren, gibt es eine Reihe von technischen Möglichkeiten. Diese richten sich oft nach der Größe der Applikation und wie diese skalieren soll. Vorgestellt werden drei Ansätze:

- ein einfacher, arraybasierter Ansatz,
- die Bibliothek Gettext und
- die Symfony Translation Komponente.

Jeder dieser Ansätze hat gewisse Vor- und Nachteile, die in den Abschnitten erläutert werden. Es ist wichtig zu verstehen, dass es nicht *den* Ansatz gibt, der immer die beste Wahl ist. Vielmehr muss das Ergebnis den Erwartungen an die Applikation

genügen. Werden etwa nur wenige Zeichenketten übersetzt, ist die Auswahl eines komplexeren Systems wahrscheinlich nicht notwendig. Soll die Applikation aber dynamisch erweiterbar sein oder sollen etwa externe Übersetzer*innen eingebunden werden, muss auf Systeme zurückgegriffen werden, die einen strukturierten und aufteilbaren Workflow unterstützen.

9.2.3 Übersetzung mit PHP-Arrays

Die einfachste Möglichkeit, Applikationen zu lokalisieren, sind PHP-Arrays. Dabei ist zu beachten, dass diese vor allem für die Übersetzung von Zeichenketten (Strings) funktionieren. Pluralbildung kann gut abgebildet werden, Dinge wie Datumsformate müssen selbständig verwaltet werden. Die komplette Verwaltung der Locales sowie die Auswahl der richtigen Strings muss selbst erfolgen. Daher eignet sich dieses System nur für sehr kleine Applikationen, die nicht erweitert werden, sprich skalieren müssen.

Aufbau der Applikation

Bei der Arbeit mit PHP-Arrays als Datenquellen für Übersetzungen empfiehlt es sich, einen eigenen Unterordner (z.B. `locale`) für die Übersetzungen zu erstellen. Darin befinden sich PHP-Dateien, die nach Sprachtags benannt sind, z.B. `locale › en.php` oder `locale › de.php`. Auch Dateien wie `locale › de-AT.php` sind möglich, wenn eine genauere Unterscheidung notwendig ist.

Jede dieser Dateien wird ein assoziatives PHP-Array enthalten, das identisch aufgebaut ist und unter bestimmten Schlüsseln die anzuzeigenden Zeichenketten in der jeweiligen Sprache enthält. Per `include` oder `require` wird die Datei für die gewünschte Locale eingebunden und somit die jeweilige Sprache ausgegeben.

Zur Demonstration wird die Datei `index.php` angelegt, die ein Formular mit einem `<select>`-Element zur Sprachauswahl enthält. Darüber hinaus noch ein Eingabefeld zur Erfassung einer Zahl, die später zur Demonstration der Pluralbildung benötigt wird. Bei den `<label>`-Elementen ist ersichtlich, dass nicht ein statischer Text, sondern der Wert eines PHP-Arrays `$messages` ausgegeben wird. Dies betrifft bereits die Übersetzung, da ja auch die Beschriftungen in der richtigen Sprache angezeigt werden sollen.

```

1 <form method="get" action="<?=$_SERVER["SCRIPT_NAME"] ?>">
2   <label for="language"><?=$messages["selectLabel"] ?></label><br>
3   <select name="locale" id="language">
4     <option value="en"><?=$messages["optionLabel"] ?></option>
5     <option value="en">English</option>
6     <option value="de">Deutsch</option>
7   </select><br>
8   <label for="messages"><?=$messages["messageLabel"] ?></label><br>
9   <input type="number" id="messages" name="nrOfMessages"><br>
10  <button type="submit"><?=$messages["sendButton"] ?></button>
11 </form>

```

Das `<select>`-Element setzt also nun unter dem Wert `locale` die Sprache. In diesem Fall entweder `en` oder `de`. Wird das Formular nun abgeschickt, wird dieser Wert mit übergeben und kann beim Anzeigen der Seite für die Lokalisierung verwendet werden. Dazu befindet sich am Anfang der PHP-Datei die Auswahl der Sprache in diesem Code-Fragment:

```

1 <?php
2
3 $locale = "en";
4 if (isset($_GET["locale"])) {
5     $locale = $_GET["locale"];
6 }
7
8 require __DIR__ . "/locale/$locale.php";

```

Zunächst wird als Standard-Locale `en` festgelegt. Diese Einstellung bestimmt auch den initialen Aufruf. Wurde aber über das Formular im `<select>`-Element ausgewählt, so wird diese Locale verwendet. Im Anschluss wird aus dem Unterverzeichnis `locale` jene PHP-Datei eingebunden, deren Name mit der Locale übereinstimmt. In der Datei ist nun das Array `$messages` definiert.

Zunächst in `locale/en.php`:

```

1 $messages = [
2     "selectLabel" => "Please select the language.",
3     "optionLabel" => "Select language",
4     "messageLabel" => "Please select the number of messages",
5     "sendButton" => "Send",
6     "welcome" => "Welcome to the site!",
7     "messages" => [
8         "zero" => "You have no messages.",
9         "one" => "You have one message.",
10        "other" => "You have %d messages.",
11    ],
12 ];

```

Dann in `locale/de.php`:

```

1 $messages = [
2     "selectLabel" => "Bitte die Sprache wählen.",
3     "optionLabel" => "Sprache wählen",
4     "messageLabel" => "Bitte die Anzahl der Nachrichten wählen.",
5     "sendButton" => "Absenden",
6     "welcome" => "Willkommen auf der Seite!",
7     "messages" => [
8         "zero" => "Du hast keine Nachrichten.",
9         "one" => "Du hast eine Nachricht.",
10        "other" => "Du hast %d Nachrichten.",
11    ],
12 ];

```

Übersetzen von Zeichenketten

Die hier hinterlegten Strings können nun auf der Seite einfach ausgegeben werden. In `$messages["selectLabel"]` befindet sich etwa der Text für die Beschriftung des `<select>`-Elements, `$messages["welcome"]` zeigt eine Begrüßung an, die unter dem Formular ausgegeben wird.

```

1 echo "<p>{$messages["welcome"]}</p>";

```

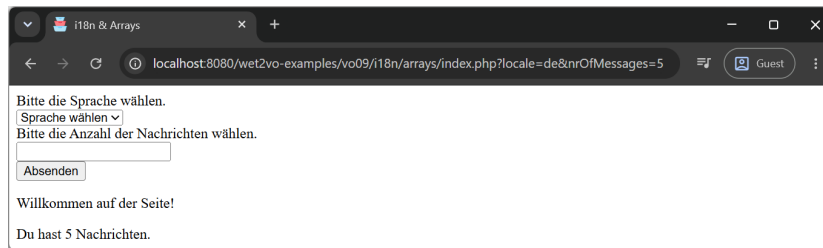


Abbildung 9.6: Im Dropdown-Menü wurde „Deutsch“ ausgewählt und die Zahl „5“ eingegeben. Das Ergebnis ist eine deutsche Seite mit einer korrekten Pluralbildung.

Pluralbildung

Das Formularfeld mit der ID `messages` erlaubt es, eine Zahl einzugeben. Basierend auf dieser soll angezeigt werden, wie viele Nachrichten ein*e Nutzer*in erhalten hat. Wird 0 eingegeben, soll (auf Deutsch) „Du hast keine Nachrichten.“ angezeigt werden, bei 1 erscheint „Du hast eine Nachricht.“ und bei jeder anderen Zahl soll „Du hast %d Nachrichten.“ ausgegeben werden, wobei %d hier für die eingegebene Zahl steht. Es benötigt also drei unterschiedliche Zeichenketten, die unter dem Schlüsselwort `messages` abgelegt sind. Unter `zero` liegt der Text für 0 Nachrichten, unter `one` der Text für eine Nachricht und unter `other` der Text für alle anderen Fälle.

Bei der Ausgabe wird nun mit einem `match`-Statement (moderne Alternative zu `switch`) basierend auf der eingegebenen Zahl entschieden, auf welchen String im Array zugegriffen wird:

```
1 if (isset($_GET["nrOfMessages"])) {
2     $nrOfMessages = $_GET["nrOfMessages"];
3
4     echo match ($nrOfMessages) {
5         "0" => $messages["messages"]["zero"],
6         "1" => $messages["messages"]["one"],
7         default => sprintf($messages["messages"]["other"], $nrOfMessages),
8     };
9 }
```

Wurde eine Zahl eingegeben wird anhand dieser entschieden. Bei der Zahl 0 wird `$messages["messages"]["zero"]` mit `echo` ausgegeben, bei der Eingabe der Zahl 1 `$messages["messages"]["one"]`. Bei jedem anderen Wert kommt der `default`-Zweig zum Zug. Hier kann nun nicht einfach mit `echo` der String ausgegeben werden, denn es muss ja die konkrete Zahl noch eingesetzt werden. Der gewählte Platzhalter `%d` kommt dabei von der PHP-Funktion `sprintf($format, $values)`¹⁰, die einen formatierten String zurückgibt. In das Muster, das in `$format` angegeben wird, können Platzhalter eingefügt werden. `%d` steht etwas für eine Ganzzahl, `%s` für einen String oder `%f` für eine Gleitkommazahl. `$values` sind die einzufügenden Werte die der Reihe nach in die Platzhalter eingefügt werden. Das Ergebnis ist der „befüllte“ String.

Das Ergebnis für die Applikation mit der Sprache Deutsch und 5 Nachrichten wird in Abbildung 9.6 dargestellt.

¹⁰<https://www.php.net/manual/de/function.sprintf.php>

Resümee und weitere Funktionalitäten

Sollen korrekte Datums-, Zeit oder Währungsformate ausgegeben werden, so muss selbst anhand von Logik entschieden werden, wie diese zu handhaben sind. Dazu müssen die jeweiligen Gegebenheiten einer Locale bekannt sein und dementsprechend umgesetzt werden. Auch gilt es zu beachten, dass die Daten in PHP-Datenstrukturen erfasst werden. Sollen externe Übersetzer*innen eingesetzt werden, so müssen diese ihre Übersetzungen in PHP-Dateien ablegen.

Somit eignet sich diese Vorgehensweise hauptsächlich dann, wenn wenige Zeichenketten übersetzt werden müssen, ansonsten aber keine weiteren Anforderungen an die Internationalisierung gestellt werden.

9.2.4 Übersetzung mit Gettext

GNU gettext – in PHP als *Gettext* verfügbar – ist eine Programmbibliothek zur Internationalisierung von Software. Sie ist ursprünglich in C verfasst und wurde in zahlreiche andere Programmiersprachen portiert, darunter auch PHP.

Grundsätzliche Funktionsweise

Der Kern der Bibliothek ist die Funktion `gettext($string)`¹¹, die als Parameter eine Zeichenkette erhält, die in der Originalsprache der Applikation angezeigt wird. Dies ist in der Regel Englisch bzw. sollte eine Sprache sein, die alle Übersetzer*innen beherrschen. Jeder String, der übersetzt werden soll, muss in dieser Funktion „verpackt“ werden. Um die Lesbarkeit zu erhöhen, steht auch der Alias `_( $string)` zur Verfügung.

Anstatt also in einer Applikation

```
1 echo "Hello World!";
```

auszugeben, wird

```
1 echo _("Hello World!");
```

verwendet.

```
1 echo gettext("Hello World!");
```

ist gleichbedeutend, aber wesentlich länger.

Vorbereitung der Umgebung

Da Gettext eine tief im Betriebssystem verankerte Bibliothek ist, arbeitet sie mit den dort installierten Locales. Um zu überprüfen, welche am Webserver installiert sind, kann `locale -a` in der Shell eingegeben werden. Dies ergibt z. B. die folgenden Locales, im `fhooe-web-dock` Docker Container sind jedoch wesentlich mehr Locales über das Debian Paket `locales-all` installiert:

`C.utf8`

`de_AT.utf8`

`de_DE.utf8`

¹¹<https://www.php.net/manual/de/function.gettext.php>

```
en_GB.utf8
en_US.utf8
POSIX
```

Um zu bestimmen, welche Locale der systemweite Standard ist, wird `locale` (ohne die Option `-a`) eingegeben. Im Docker Container ist `en_US.utf8` überall als Standard hinterlegt.

```
LANG=en_US.utf8
LANGUAGE=en_US.utf8
LC_CTYPE="en_US.utf8"
LC_NUMERIC="en_US.utf8"
LC_TIME="en_US.utf8"
LC_COLLATE="en_US.utf8"
LC_MONETARY="en_US.utf8"
LC_MESSAGES="en_US.utf8"
LC_PAPER="en_US.utf8"
LC_NAME="en_US.utf8"
LC_ADDRESS="en_US.utf8"
LC_TELEPHONE="en_US.utf8"
LC_MEASUREMENT="en_US.utf8"
LC_IDENTIFICATION="en_US.utf8"
LC_ALL=en_US.utf8
```

Vorbereitung der Applikation

Wie bereits in Abschnitt 9.2.3 gezeigt, soll eine kleine Applikation geschrieben werden, die mit einem `<select>`-Element die Sprache wechseln und einen Singular- und Plural-String anzeigen kann.

Die Datei `index.php` enthält also wieder ein Formular. Dieses Mal sind alle zu übersetzenden Zeichenketten mit der Gettext-Syntax gekennzeichnet. Die Werte der `<option>`-Elemente sind nun Locales, wie sie im Linux-Betriebssystem soeben installiert bzw. vorgefunden wurden.

```
1 <form method="get" action="<?=$_SERVER["SCRIPT_NAME"] ?>">
2   <label for="language"><?=$__("Please select the language.") ?></label><br>
3   <select name="locale" id="language">
4     <option value="en_US.utf8"><?=$__("Select language") ?></option>
5     <option value="en_US.utf8">English</option>
6     <option value="de_AT.utf8">Deutsch</option>
7   </select><br>
8   <label for="messages"><?=$__("Please select the number of messages.") ?></label><br>
9   <input type="number" id="messages" name="nrOfMessages"><br>
10  <button type="submit"><?=$__("Send") ?></button>
11 </form>
```

Nach dem Formular wird ein String zur Begrüßung ausgegeben:

```
1 echo "<p>" . _("Welcome to the site!") . "</p>";
```

Am Beginn der PHP-Datei findet nun wieder die Auswahl der Sprache statt. Zusätzlich werden die Einstellungen für Gettext vorgenommen:

```

1 $locale = "en_US.utf8";
2 if (isset($_GET["locale"])) {
3     $locale = $_GET["locale"];
4 }
5
6 putenv("LANG=$locale");
7 setlocale(LC_ALL, $locale);
8 $domain = "messages";
9 bindtextdomain($domain, __DIR__ . "/locale");
10 textdomain($domain);
11 bind_textdomain_codeset($domain, "UTF-8");

```

Als Locale wird `en_US.utf8` angenommen. Dies ist die im System verwendete Locale. Wiederum kann über das Formular etwas anderes gesetzt werden.

Danach finden einige Einstellungen statt. Zunächst wird in Zeile 6 eine Umgebungsvariable für die Sprache gesetzt, in Zeile 7 folgt dann die Locale für die eingebauten PHP Internationalisierungsfunktionen. `LC_ALL` bedeutet, dass eine Locale für alle Features (von Stringvergleichen über Währungsformatierung bis hin zu Zahlen- und Datumsformatierung) verwendet wird. In den Zeilen 8 und 9 wird die Domäne angegeben. Sie bestimmt später den Namen der Dateien, welche die übersetzten Strings beinhalten werden. Diese Domain wird dann einem Verzeichnis, in diesem Fall dem Unterordner `locale` zugewiesen. Dort müssen diese Dateien dann abgespeichert werden. Schließlich wird die Domäne in Zeile 10 aktiviert und schließlich in Zeile 11 die Zeichenkodierung für die Domäne definiert. Nun können die Dateien mit den Übersetzungen erstellt werden.

Erstellung der Übersetzungen

Gettext verwendet zur Übersetzung sogenannte *Portable Objects* bzw. PO-Dateien. Damit diese korrekt gefunden werden, muss eine korrekte Dateistruktur angelegt werden. Der oberste Ordner `locale` wurde in PHP bereits spezifiziert. Darunter muss für jede Sprache aber eine festgelegte Struktur erzeugt werden. Die Übersetzungsdateien für `de_AT` müssen im Verzeichnis `locale▶de_AT▶LC_MESSAGES` liegen. Der Dateiname muss mit der angegebenen Domain übereinstimmen, also `messages.po` lauten.

Nun können mit den Commandline-Tools `xgettext` und `msginit` zunächst ein PO-Template (`.pot`-Datei) und daraus eine PO-Datei erstellt werden. In diese PO-Datei werden dann die übersetzten Strings eingetragen. Schließlich wird die PO-Datei mittels `msgfmt` in ein Machine Object (MO-Datei) umgewandelt, die die übersetzten Texte in einem Binärformat enthält und die zur Laufzeit verwendet werden.

Um diesen Prozess zu Vereinfachen kann auch ein grafisches Werkzeug verwendet werden. Das bekanntest ist Poedit¹². Mit ihm wird zunächst eine leere POT-Datei angelegt, dann wird Poedit mitgeteilt, wo sich die Quelldateien befinden. Aus diesen liest das Werkzeug die zu übersetzenden Strings aus und erzeugt die PO-Datei. In der grafischen Oberfläche werden die Übersetzungen eingetragen und gespeichert. Dies erzeugt im Hintergrund die passende MO-Datei `messages.mo`. Abbildung 9.7 stellt die Oberfläche von Poedit dar.

¹²<https://poedit.net/>

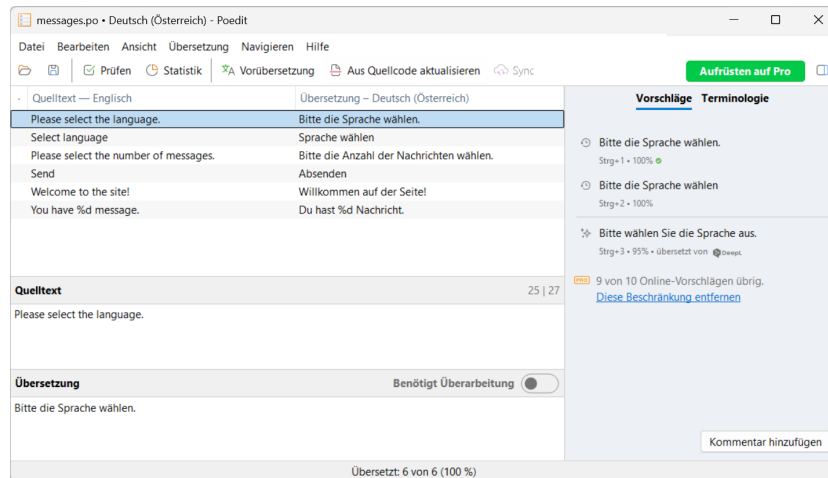


Abbildung 9.7: Poedit zeigt alle zu übersetzenden Strings zusammen mit der Übersetzung an und generiert alle notwendigen Dateien.

Die entstandene PO-Datei enthält den Originalstring zusammen mit seiner Übersetzung und einigen Metadaten.

```

1 #: index.php:24
2 msgid "Please select the language."
3 msgstr "Bitte die Sprache wählen."
4
5 #: index.php:26
6 msgid "Select language"
7 msgstr "Sprache wählen"
8
9 #: index.php:30
10 msgid "Please select the number of messages."
11 msgstr "Bitte die Anzahl der Nachrichten wählen."
12
13 #: index.php:32
14 msgid "Send"
15 msgstr "Absenden"
16
17 #: index.php:36
18 msgid "Welcome to the site!"
19 msgstr "Willkommen auf der Seite!"

```

Nachdem die MO-Dateien generiert wurden, kann die Applikation ausgeführt werden. Die Strings werden korrekt übersetzt.

MO-Dateien werden vom Betriebssystem gecacht, um maximale Performance zu bieten. Werden die PO-Dateien editiert und neue PO-Dateien generiert, muss der Apache Webserver neu gestartet werden (unter Debian im Docker Container etwa mittels `service apache2 restart`), damit die Änderungen aus dem Dateisystem übernommen werden. Ein Aufruf von `clearstatcache()`; am Ende des PHP-Skripts, kann während der Entwicklungsphase diesen Cache löschen und einen Restart vermeiden. Dies funktioniert jedoch nicht immer.

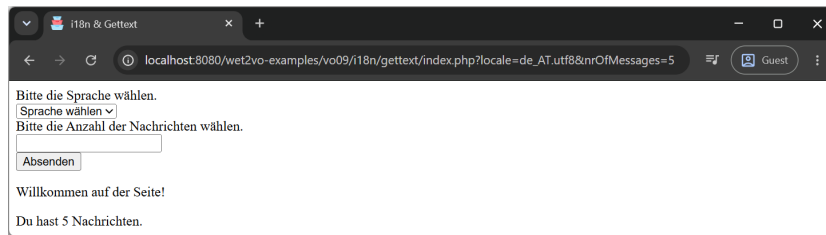


Abbildung 9.8: Im Dropdown-Menü wurde „Deutsch“ ausgewählt und die Zahl „5“ eingegeben. Das Ergebnis ist eine deutsche Seite mit einer korrekten Pluralbildung.

Pluralbildung

Für die Pluralbildung ist die Funktion `ngettext($singular, $plural, $count)`¹³ verfügbar. Das erste Argument ist die Einzahl, das zweite die Mehrzahl. Das dritte Argument dient Gettext festzustellen, welche Form gewählt werden muss.

Im Beispiel wird dies genutzt, um eine einfache Form der Benachrichtigung über neue Nachrichten zu erstellen.

```
1 if (isset($_GET["nrOfMessages"])) {
2     $nrOfMessages = intval($_GET["nrOfMessages"]);
3
4     echo sprintf(ngettext("You have %d message.", "You have %d messages.",
5                           $nrOfMessages), $nrOfMessages);
5 }
```

Die Einzahl „You have %d message.“ mit der Mehrzahl „You have %d messages.“ zusammen mit der Anzahl der Nachrichten aus dem Formular an `ngettext()` übergeben. In der PO-Datei wird mit Poedit eine Übersetzung angelegt. Diese erzeugt folgenden Eintrag:

```
1 #: index.php:41
2 #, php-format
3 msgid "You have %d message."
4 msgid_plural "You have %d messages."
5 msgstr[0] "Du hast %d Nachricht."
6 msgstr[1] "Du hast %d Nachrichten."
```

Je nachdem, welche Anzahl in `$nrOfMessages` enthalten wird, wählt Gettext die richtige Form aus und gibt sie als String zurück. Ist deutsch aktiviert, etwa wie folgt:

- 0: Du hast %d Nachrichten.
- 1: Du hast %d Nachricht.
- 2: Du hast %d Nachrichten.

Über `sprintf()` wird dann %d durch die eigentliche Zahl ersetzt, sodass eine passende Nachricht ausgegeben wird. Für 5 Nachrichten sieht dies wie in Abbildung 9.8 dargestellt aus.

¹³<https://www.php.net/manual/de/function.ngettext.php>

Resümee und weitere Funktionalitäten

Gettext ist ein langjähriger Standard über Übersetzungen in Programmen. Das System der PO-Dateien ist weit verbreitet, es gibt viele Werkzeuge, auch auf kommerzieller Ebene, die damit arbeiten. Nachdem die Applikation mit den Gettext-Funktionen erstellt und die PO-Dateien generiert wurden, können diese Dateien an externe Übersetzer*innen weitergegeben werden, diese können völlig unabhängig von der Applikation die Zeichenketten übersetzen und die Dateien retournieren. Wissen über PHP oder den Aufbau der Applikation sind dabei nicht notwendig.

Gettext hat jedoch keinerlei Werkzeuge oder Funktionalitäten, um etwa mit Datum, Zeit oder Währungen zu arbeiten. Diese Dinge müssen selbst verwaltet werden. Darüber hinaus ist die Installation nicht immer einfach und überall möglich. Auch bedeutet die enge Integration ins Betriebssystem, dass etwas an Flexibilität verloren geht, vor allem, wenn die Applikation auf anderen Betriebssystemen laufen soll bzw. Betriebssysteme nicht für den Einsatz mit Gettext korrekt konfiguriert sind. Fehlende Locales müssen etwa nachinstalliert werden. Fehlen hierfür die Rechte, kann die Applikation mit diesen Sprachen nicht funktionieren.

Gettext hat seine Stärken jedoch vor allem bei Dingen wieder Pluralbildung, wo die Bibliothek selbst die Auswahl der korrekten Formen übernimmt.

Somit eignet sich Gettext immer dann, wenn die Übersetzung im Vordergrund steht. Durch die gute Toolunterstützung kann die Übersetzung dabei schnell und effizient erfolgen.

9.2.5 Übersetzung mit symfony/translation

Die dritte Möglichkeit zur Internationalisierung und Lokalisierung von Applikationen bieten Komponenten. Viele der großen Frameworks beinhalten Internationalisierungskomponenten, die auch einzeln verwendbar sind. Eine solche ist `symfony/translation` aus dem Symfony Framework.

Installation und Einbindung

Die Komponente wird zunächst über Composer installiert:

```
1 $ composer require symfony/translation
```

Bei der Einbindung wird, wie bereits von anderen Komponenten bekannt, der Composer-Autoloader am Beginn der Datei eingebunden und zunächst ein `Translator`-Objekt angelegt. Diesem wird die Locale übergeben, in die es übersetzen soll. In dem Beispiel soll die Locale wieder über ein `<select>`-Menü gesetzt werden können.

```
1 use Symfony\Component\Translation\Loader\JsonFileLoader;
2 use Symfony\Component\Translation\Translator;
3
4 require "vendor/autoload.php";
5
6 $locale = "en-US";
7 if (isset($_GET["locale"])) {
8     $locale = $_GET["locale"];
9 }
10
11 $translator = new Translator($locale);
```

Nachdem das `Translator`-Objekt erzeugt wurde, müssen die Ressourcen mit den übersetzten Zeichenketten hinzugefügt werden. `symfony/translation` unterstützt dabei eine Reihe von Loader-Klassen, mit denen diese Daten dem Translator übergeben werden können. Die Klasse kann JSON-, YAML-, XML-, PO-, MO- oder einfache PHP-Dateien laden. Zusätzlich gibt es einen Array-Loader und auch aus Datenbanken können Übersetzungen geladen werden. Für das folgende Beispiel kommt ein JSON-Loader zum Einsatz. Die Übersetzungen sind dabei in einem JSON-Objekt hinterlegt, das die Strings eindeutigen Schlüsseln zuordnet.

Zunächst wird ein Objekt der Klasse `JsonFileLoader` angelegt und dem Translator unter dem Namen `json` hinzugefügt. Danach werden zwei Ressourcen – die JSON-Dateien mit den Übersetzungen – angegeben. Beim Hinzufügen mit `addResource()` wird der Name des Loaders (`json`), der Pfad zur Datei (`translations/messages.de.json`), sowie die Locale der Datei (d.h. die Sprache der enthaltenen Strings) angegeben.

Wichtig: `symfony/translation` nimmt an, dass die Datei den Namen `messages`, gefolgt von einem Sprachtag und der Dateiendung trägt. `messages` ist hierbei die sogenannte Domäne, unter der die Übersetzungen verfügbar sind (ähnlich wie bei Gettext). Soll ein anderer Name verwendet werden, so muss die geänderte Domäne als vierter Parameter bei `addResource()` angegeben werden (`message` wird als Standard angenommen und muss nicht extra angeführt werden).

```
1 $translator->addLoader("json", new JsonFileLoader());
2 $translator->addResource("json", "translations/messages.de.json", "de-AT");
3 $translator->addResource("json", "translations/messages.en.json", "en-US");
```

Vorbereitung der Applikation und Übersetzungen

Die Applikation besteht wieder aus einem Formular, in dem in einem `<select>`-Element die Sprache und in einem Eingabefeld die Anzahl von Nachrichten eingegeben werden können. Darunter wird eine Willkommensnachricht ausgegeben.

```
1 <form method="get" action="<?=$_SERVER["SCRIPT_NAME"] ?>">
2   <label for="language"><?=$translator->trans("selectLabel") ?></label><br>
3   <select name="locale" id="language">
4     <option value="en-US"><?=$translator->trans("optionLabel") ?></option>
5     <option value="en-US">English</option>
6     <option value="de-AT">Deutsch</option>
7   </select><br>
8   <label for="messages"><?=$translator->trans("messageLabel") ?></label><br>
9   <input type="number" id="messages" name="nrOfMessages"><br>
10  <button type="submit"><?=$translator->trans("sendButton") ?></button>
11 </form>
12 <?php
13
14 echo "<p>{<?=$translator->trans("welcome")}</p>";
```

Jede Zeichenkette, die übersetzt werden soll, wird in einen `$translator->trans()` Aufruf verpackt. Übergeben wird ein Schlüsselwort, unter dem die Übersetzungen in den verschiedenen JSON-Dateien hinterlegt sind. `selectLabel` beinhaltet den Label-Text für das `<select>`-Element, unter `welcome` ist der Begrüßungstext hinterlegt. Die JSON-Dateien dazu sehen wie folgt aus:

`translations/messages.en.json`:

```

1 {
2   "selectLabel": "Please select the language.",
3   "optionLabel": "Select language",
4   "messageLabel": "Please select the number of messages",
5   "sendButton": "Send",
6   "welcome": "Welcome to the site!",
7   "messages": {
8     "zero": "You have no messages.",
9     "one": "You have one message.",
10    "other": "You have %d messages."
11  }
12 }

```

translations/messages.de.json:

```

1 {
2   "selectLabel": "Bitte die Sprache wählen.",
3   "optionLabel": "Sprache wählen",
4   "messageLabel": "Bitte die Anzahl der Nachrichten wählen.",
5   "sendButton": "Absenden",
6   "welcome": "Willkommen auf der Seite!",
7   "messages": {
8     "zero": "Du hast keine Nachrichten.",
9     "one": "Du hast eine Nachricht.",
10    "other": "Du hast %d Nachrichten."
11  }
12 }

```

Bei der Ausführung wählt das **Translator**-Objekt nun die Strings aus der jeweiligen Locale und zeigt diese an.

Pluralbildung

Für die Demonstration der Pluralbildung wurde der Eintrag **messages** in den JSON-Dateien angelegt. Er enthält ein Unterobjekt mit den drei Schlüsseln **zero**, **one** und **other** in denen die entsprechenden Formulierungen für keine Nachrichten, eine Nachricht und mehr als eine Nachricht hinterlegt sind.

Die Logik dazu wird in der PHP-Datei anhand des Wertes **nrOfMessages** aus dem Formular gesteuert. Je nach Wert wird einer der drei Werte in einem **match**-Statement ausgegeben. Bei der Position **other** wird zusätzlich der Parameter **%d** mit dem Wert aus **\$nrOfMessages** ersetzt. Dies ist über die Angabe eines zweiten Parameters bei **trans()** möglich.

```

1 if (isset($_GET["nrOfMessages"])) {
2   $nrOfMessages = $_GET["nrOfMessages"];
3
4   echo match ($nrOfMessages) {
5     "0" => $translator->trans("messages.zero"),
6     "1" => $translator->trans("messages.one"),
7     default => $translator->trans("messages.other", ["%d" => $nrOfMessages])
8   };
9 }

```

Abbildung 9.9 zeigt die Anwendung, nachdem Deutsch ausgegeben und die Zahl 5 ausgewählt wurde.

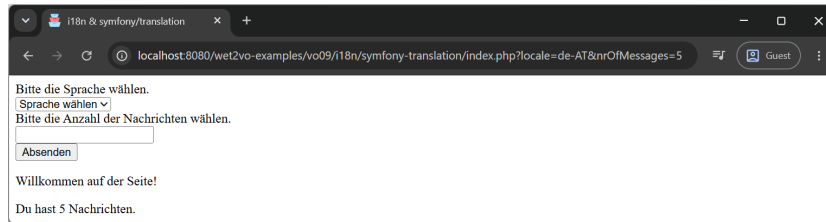


Abbildung 9.9: Im Dropdown-Menü wurde „Deutsch“ ausgewählt und die Zahl „5“ eingegeben. Das Ergebnis ist eine deutsche Seite mit einer korrekten Pluralbildung.

Übersetzung in Latte Templates

Die `symfony/translation`-Komponente lässt sich nahtlos in die Latte Template-Engine integrieren. Latte bietet hierfür native Übersetzungs-Tags in der Form `{_ 'key' }`. Um diese Funktionalität zu aktivieren, muss die `TranslatorExtension` von Latte registriert und mit dem Symfony-Translator verknüpft werden.

Zunächst wird in `index.php` das `Translator`-Objekt mit einem Loader und den Übersetzungs-Ressourcen wie zuvor gezeigt angelegt. Danach wird die Latte-Engine wie etwa aus Vorlesung 6 bekannt, erzeugt. Schließlich wird mittels `addExtension()` eine Instanz der `TranslatorExtension` hinzugefügt. Sie erwartet als Parameter ein `Callable`, also etwas, das sie aufrufen kann. Hier könnte etwa eine anonyme Funktion (eine sogenannte Closure) hinterlegt werden – die bereits bekannte First-Class-Callable-Syntax ist hier jedoch wesentlich eleganter und kürzer. Sie reicht die Parameter aus dem Template direkt an die `symfony/translation`-Komponente weiter.

```

1 $locale = "en-US";
2 if (isset($_GET["locale"])) {
3     $locale = $_GET["locale"];
4 }
5
6 $translator = new Translator($locale);
7 $translator->addLoader("json", new JsonFileLoader());
8 $translator->addResource("json", "translations/messages.de.json", "de-AT");
9 $translator->addResource("json", "translations/messages.en.json", "en-US");
10
11 $latte = new Engine();
12 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
13 $latte->setCacheDirectory(__DIR__ . "/cache");
14
15 $latte->addExtension(new TranslatorExtension($translator->trans(...)));

```

Nun kann eine Template-Datei `index.latte` angelegt werden. Diese enthält wieder ein Formular zur Sprachauswahl und zur Eingabe einer Zahl, gibt eine Willkommensnachricht und die Anzahl der Nachrichten (zur Demonstration der Pluralbildung) aus:

```

1 <!DOCTYPE html>
2 <html lang="{ $locale }">
3   <head>
4     <meta charset="UTF-8">
5     <title>i18n & symfony/translation + Latte Templates</title>
6   </head>
7   <body>

```

```

8     <form method="get" action="{$_scriptName}">
9         <label for="language">{$_selectLabel}</label><br>
10        <select name="locale" id="language">
11            <option value="en-US">{$_optionLabel}</option>
12            <option value="en-US">English</option>
13            <option value="de-AT">Deutsch</option>
14        </select><br>
15        <label for="messages">{$_messageLabel}</label><br>
16        <input type="number" id="messages" name="nrOfMessages"><br>
17        <button type="submit">{$_sendButton}</button>
18    </form>
19
20    <p>{$_welcome}</p>
21
22    {if $nrOfMessages != null}
23        <p n:if="$nrOfMessages == 0">{$_messages.zero}</p>
24        <p n:elseif="$nrOfMessages == 1">{$_messages.one}</p>
25        <p n:else>{$_messages.other, ['%d' => $nrOfMessages]}</p>
26    {/if}
27 </body>
28 </html>

```

Alle Zeichenketten, die übersetzt werden sollen, werden mit dem Schlüssel unter dem sie in den Übersetzungs-Ressourcen zu finden sind, innerhalb des `{_}`-Tags angegeben. Dies weist die `Translator`-Klasse an, den jeweiligen String in der aktiven Locale an dieser Stelle einzusetzen. Um bei der Pluralbildung Parameter mitzugeben, wird dem Tag einfach ein assoziatives Array übergeben (hier `['%d' => $nrOfMessages]`).

In der Hauptdatei muss nun nur noch das Template gerendert werden:

```

1 $latte->render("index.latte", [
2     "locale" => $locale,
3     "scriptName" => $_SERVER["SCRIPT_NAME"],
4     "nrOfMessages" => isset($_GET["nrOfMessages"]) ? intval($_GET["nrOfMessages"]) :
        null,
5 ]);

```

Die einzigen Platzhalter, die übergeben werden müssen sind die, die nicht übersetzt werden. Dies sind der Wert des `action`-Attributs des Formulars sowie der eingegebene Wert aus dem Formularfeld. Ist dieser nicht vorhanden (beim ersten Aufruf) wird `null` übergeben. Zudem wird der Locale-String übergeben, um das `lang`-Attribut beim `<html>`-Element korrekt setzen zu können.

Übersetzung mit dem ICU MessageFormat

Alle bisher gezeigten Übersetzungen arbeiteten mit weitgehend statischen Zeichenketten. Maximal wurde an einer Stelle (den Nachrichten) ein Wert eingesetzt. Oft jedoch sind Strings komplexer und ergeben sich aus mehreren Bedingungen, die verschiedene Werte wie Namen, Zahlen oder ein Datum enthalten. Da die Wortfolge hier in unterschiedlichen Sprachen meist völlig unterschiedlich ist, bleibt meist nur die Aufteilung in mehrere Substrings, die einzeln übersetzt und wieder zusammengefügt werden müssen. Dies verkompliziert jedoch Übersetzungen, da die Zusammenhänge oft nicht mehr gegeben sind. Daher muss versucht werden, einen gesamten String zu erzeugen, der je nach Sprache anders zusammengesetzt ist und diese variablen Werte enthält.

Tabelle 9.4: Beispiele für ICU MessageFormat Muster. Für die Beispielausgabe wird die Locale `de-AT` angenommen.

Muster	Werte	Resultat
Hallo {name}!	<i>name</i> : Wolfgang	Hallo Wolfgang!
Das kostet {value,number,currency}.	<i>value</i> : 3	Das kostet € 3,00.
Heute ist der {datum,date,short}	<i>datum</i> : <code>new DateTime()</code>	Heute ist der 02.06.26
Es ist jetzt {uhrzeit,time}	<i>uhrzeit</i> : <code>new DateTime()</code>	Es ist jetzt 21:02:57

Dies ist mit Hilfe des ICU MessageFormats Möglich. ICU steht für „International Components for Unicode“ und stellt ein Set an C/C++ und Java-Bibliotheken dar, die bei der Internationalisierung von Applikationen verwendet werden können. In PHP steht MessageFormat über die Klasse `MessageFormatter`¹⁴ zur Verfügung und kann verwendet werden, um sprachneutrale Strings zu erzeugen.

Ein solcher String besteht aus einem Muster, ähnlich wie bei PDO Prepared Statements. Variable Inhalte wie Zahlen, Daten, wechselnde adjektive oder Anreden werden über Platzhalter (Argumente) repräsentiert und in geschwungen Klammern angegeben. Steht nur der Platzhalter, wird dieser unformatiert ausgegeben. Zusätzlich können mit Beistrich getrennt noch Formatierungsangaben gemacht werden. Zur Verfügung stehen `number`, `date` und `time` und Zusatzargumente zur detaillierten Formatierung¹⁵. Bei der Ausgabe werden die Platzhalter dann befüllt und der fertige String wird ausgegeben. Tabelle 9.4 zeigt einige Beispiele.

Neben diesen simplen Argumenten gibt es auch komplexe. Erwähnenswert sind `plural` und `select`.

Plural ermöglicht es, Strings basierend auf einem numerischen Wert zusammen mit Pluralregeln für eine Sprache zu selektieren. Über die Kategorien `zero`, `one`, `two`, `few`, `many` und `other` können – je nach Sprache – die Pluralregeln abgebildet werden. Für Deutsch werden in der Regel nur `zero`, `one` und `other` benötigt:

```
1 {nr,plural,
2   zero {Niemand kam.}
3   one {Jemand kam.}
4   other {Viele kamen.}
5 }
```

Wird nun für `nr` etwa 5 übergeben, wird „Viele kamen.“ ausgewählt. Bei 1 „Jemand kam“.

Ähnlich verhält sich `select`. Hier erfolgt die Auswahl aufgrund von selbst festlegbaren Schlüsselwörtern – lediglich die Fallback-Option `other` muss existieren. `select` ist für die Auswahl des Geschlechts hilfreich:

```
1 {gender,select,
2   male {Er}
3   female {Sie}
4   other {They}
```

¹⁴<https://www.php.net/manual/de/class.messageformatter.php>

¹⁵Siehe dazu <https://unicode-org.github.io/icu-docs/apidoc/released/icu4j/com/ibm/icu/text/MessageFormat.html> für eine vollständige Auflistung der Parameter.


```
5 }
```

Wird *male* übergeben, so wird „Er“ ausgegeben, bei *female* „Sie“ und bei jedem anderen Wert, z. B. *divers* oder *other*, wird „They“ ausgewählt.

Der Online ICU Message Editor¹⁶ erlaubt ein einfaches Experimentieren mit Mustern und den zu befüllenden Werten.

Die `symfony/translate`-Komponente unterstützt den Umgang mit dem ICU MessageFormat. Um dem `Translator`-Objekt mitzuteilen, dass die enthalten Zeichenketten in diesem Format vorliegen, müssen Dateinamen und Domäne mit dem Suffix `+intl-icu` ergänzt werden:

```
1 $translator = new Translator("de-AT");
2 $translator->addLoader("yaml", new YamlFileLoader());
3 $translator->addResource("yaml", "translations/messages+intl-icu.en.yaml", "en-US",
   "messages+intl-icu");
4 $translator->addResource("yaml", "translations/messages+intl-icu.de.yaml", "de-AT",
   "messages+intl-icu");
```

Da ICU MessageFormat Strings besser lesbar sind, wenn sie mit Zeilenumbrüchen formatiert werden, bietet sich hier YAML als Format für die Übersetzungs-Ressourcen an, denn JSON erlaubt in den Zeichenketten keine Zeilenumbrüche (bzw. nur durch Angabe von `\n`).

Um YAML verwenden zu können, muss die Komponente `symfony/yaml` mit Composer installiert werden:

```
1 $ composer require symfony/yaml
```

Zum Laden der Ressourcen kommt nun die Klasse `YamlFileLoader` zum Einsatz. Die Dateien enthalten Strings zu einer Party-Einladung, basierend auf der einladenden Person, dem Gast, der Anzahl der Gäste und dem Datum und sehen wie folgt aus:

`translations>messages+intl-icu.en.yaml`

```
1 ---
2 party: |-
3 {gender_of_host, select,
4   female {
5     {num_guests, plural, offset:1
6       =0 {{host}} does not give a party on {party_date, date}.}
7       =1 {{host}} invites {guest} to her party on {party_date, date}.}
8       =2 {{host}} invites {guest} and one other person to her party on {party_date,
9         date}.}
9     other {{host}} invites {guest} and # other people to her party on {party_date,
10      date}.}
10   }
11 }
12 male {
13   {num_guests, plural, offset:1
14     =0 {{host}} does not give a party on {party_date, date}.}
15     =1 {{host}} invites {guest} to his party on {party_date, date}.}
16     =2 {{host}} invites {guest} and one other person to his party on {party_date,
17       date}.}
17   other {{host}} invites {guest} and # other people to his party on {party_date,
18     date}.}
18 }
```

¹⁶<https://format-message.github.io/icu-message-format-for-translators/editor.html>

```

19   }
20   other {
21     {num_guests, plural, offset:1
22       =0 {{host}} does not give a party on {party_date, date}.}
23       =1 {{host}} invites {guest} to their party on {party_date, date}.}
24       =2 {{host}} invites {guest} and one other person to their party on {party_date,
25         date}.}
26     other {{host}} invites {guest} and # other people to their party on {party_date
27       , date}.}
28   }

```

translations>messages+intl-icu.de.yaml

```

1 ---
2 party: |-
3 {gender_of_host, select,
4   female {
5     {num_guests, plural, offset:1
6       =0 {{host}} veranstaltet keine Party am {party_date, date}.}
7       =1 {{host}} lädt {guest} zur ihrer Party am {party_date, date} ein.}
8       =2 {{host}} lädt {guest} und eine weitere Person zu ihrer Party am {party_date,
9         date} ein.}
10    other {{host}} lädt {guest} und # andere Personen zu ihrer Party am {party_date
11      , date} ein.}
12  }
13  }
14  male {
15    {num_guests, plural, offset:1
16      =0 {{host}} veranstaltet keine Party am {party_date, date}.}
17      =1 {{host}} lädt {guest} zur seiner Party am {party_date, date} ein.}
18      =2 {{host}} lädt {guest} und eine weitere Person zu seiner Party am {party_date
19        , date} ein.}
20    other {{host}} lädt {guest} und # andere Personen zu seiner Party am {
21      party_date, date} ein.}
22  }
23  }
24  other {
25    {num_guests, plural, offset:1
26      =0 {{host}} veranstaltet keine Party am {party_date, date}.}
27      =1 {{host}} lädt {guest} zur their Party am {party_date, date} ein.}
28      =2 {{host}} lädt {guest} und eine weitere Person zu their Party am {party_date,
29        date} ein.}
30    other {{host}} lädt {guest} und # andere Personen zu their Party am {party_date
31      , date} ein.}
32  }
33  }

```

Zunächst wird auf der obersten Ebene mit einem `select`-Argument je nach Geschlecht des*der Gastgebers*Gastgeberin (`host`) entschieden. Danach folgt die jeweilige Pluralbildung je nach Anzahl der Gäste. Bei 0 wird keine Party veranstaltet, bei 1 eine Party mit der in `guest` genannten Person und bei 2 und `other` eine Party mit der genannten Person und weiteren. `party_date` gibt das Datum der Party im jeweils lokalisierten Format aus.

Nun wird der Translator aufgerufen und die Argumente werden mit übergeben:



Abbildung 9.10: Durch die übergebenen Parameter entsteht ein einzelner String, der grammatikalisch korrekt und nach den Regeln der Locale (hier **de-AT**) formatiert ist.

```
1 $translator->trans("party", [  
2     "host" => "Wolfgang",  
3     "gender_of_host" => "male",  
4     "guest" => "John Doe",  
5     "num_guests" => 3,  
6     "party_date" => new DateTime(),  
7 ])
```

Dieser Aufruf erzeugt die Ausgabe in Abbildung 9.10.

Resümee und weitere Funktionalitäten

Die `symfony/translation`-Komponente ist sicherlich die kompletteste Lösung zur Internationalisierung. Sie ermöglicht einfaches Übersetzen von Zeichenketten genauso wie die modulare Verwendung des ICU MessageFormats. Sie deckt damit auch Datum, Uhrzeit, Währung und viele andere Dinge ab. Auch das Zusammenspiel mit Twig Templates ist ein Argument für die Verwendung.

Die Komponente kann noch einige Dinge mehr, die unter <https://symfony.com/doc/current/translation.html> nachgelesen werden können.

9.3 Weiterführende Literatur

Die Arbeit mit Datum und Zeit gehört zum Standardrepertoire von PHP und ist daher in den meisten modernen Büchern gut abgedeckt. Erwähnt seien dabei wie immer [4, 5].

Literatur zur Internationalisierung und Lokalisierung ist jedoch kaum vorhanden. Alle großen und bekannten PHP-Bücher sparen dieses Thema gekonnt aus. Somit kann nur auf [3] verwiesen werden. Die Dokumentation enthält ein eigenes Kapitel zu allen Internationalisierungsfunktionen und zeigt Beispiele, wie sie eingesetzt werden können. Tutorials und in den Weiten des WWW können dies noch ergänzen.

Quellenverzeichnis

- [1] Richard Ishida und Susan K. Miller. *Lokalisierung vs. Internationalisierung*. Übers. von Jens Oliver Meiert. 2. Nov. 2018. URL: <https://www.w3.org/International/questions/qa-i18n.de.html> (besucht am 31. 05. 2026) (siehe S. 9-7).
- [2] Addison Phillips und Mark Davis. *Tags for Identifying Languages*. RFC 5646. IETF, Sep. 2009, S. 1–83. DOI: 10.17487/RFC5646 (siehe S. 9-7).
- [3] The PHP Documentation Group. *PHP-Handbuch*. 31. Mai 2026. URL: <https://www.php.net/manual/de/> (siehe S. 9-26).
- [4] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 9-26).
- [5] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 9-26).